

Linux 课程学习总结

收获 · 感想 · 建议

赵熠卓

学号：2023210314

2026 年 5 月 31 日

SHA256: 7d4b9a24a0611ade2dd7ed4bb1229ff7d83298c5f0468789076f2374cc471aa8

1. 课程收获
2. 课程感想
3. 课程建议

课程收获

包管理模式

- 掌握了 Ubuntu/Debian 体系的包管理：apt 负责在线源管理，dpkg 处理本地包
- apt update/upgrade 更新索引与升级系统，install/remove/purge 管理软件生命周期
- search/show/depends 快速查询包信息与依赖关系——不再需要打开网页搜索
- 配置清华镜像源时了解了 DEB822 格式，理解了 /etc/apt/sources.list.d/ 的结构
- 遇到过一次 dpkg --configure -a 修复依赖断裂——从报错到解决的过程让我对 dpkg 底层机制有了切身体会
- 还顺带配置了 oh-my-zsh：换主题、加插件（git、z、syntax-highlighting），终端体验大幅提升

- 一开始只知道 r= 读 / w= 写 / x= 执行，实验后才发现 r/w/x 对文件和目录的语义完全不同
- 对目录而言：r 可列出内容但无法 cd，x 可进入但无法 ls——这两者独立控制
- 作业中手动创建了 000 到 777 全部 8 种文件 + 8 种目录，逐个测试——Permission denied 出现的位置比书上说的要复杂
- **SUID/SGID** 是这次学习的新概念：/usr/bin/passwd 和 /usr/bin/su 上有 SUID 位，普通用户借此临时获得 root 权限
- 这让我意识到 Linux 安全不是靠「禁止」，而是靠精确的权限机制设计——非常精巧

- 入门前只会 i 进入插入模式、:wq! 强制保存退出，甚至连怎么删除一行都不知道
- 20 项渐进练习后，掌握了：h/j/k/l 导航、w/b 按词跳转、dd/yy/p 剪切复制粘贴、u 撤销
- 更高级的：Visual 模式多行选择、q 宏录制批量操作、/ 和 :s/%s 搜索替换
- 最重要的是发现了 **Cheat Sheet** 这个东西——一张图涵盖了常用键位，贴在显示器旁边比反复查 man 高效太多
- 现在日常用 Neovim + LazyVim 写 Markdown 和 Shell 脚本，基本实现了不离开键盘的编辑体验

Shell 深入理解

- 环境变量方面：\$HOME、\$PATH、\$?（上条命令返回值）、\$LANG 等在作业中反复使用
- 通过设置 PS1 自定义了终端 prompt——颜色、用户名、当前路径一目了然，不再只是单调的 \$
- 理解了 source script.sh 和 ./script.sh 的关键区别：前者在当前 **Shell** 中执行（变量会保留），后者启动子 Shell（变量随子进程消亡）
- 工具链从作业 06/07 中学习了很多：tr 字符转换、grep 模式过滤、sort | uniq -c 排序聚合
- 课外还自学了：jq 处理 JSON、whatis 快速看命令用途、man 深入查阅、su 切换用户
- RFC 2460 词频统计的管道组合和 Ramanujan 公式计算 PI（bc -l 200 位精度），是两段印象最深的操作

- 通过本课程首次系统接触 LaTeX 排版——本篇总结报告就是用 Beamer 写的
- **Beamer**: 理解了 frame 的概念、section 分章节、`\metroset` 定制主题样式
- 用 `ctex` 宏包解决了中文排版, `minted` 做代码高亮, `booktabs` 画专业表格
- **cetz**: 尝试了用 `cetz` 宏包在 LaTeX 里画流程图, 比外部绘图工具更精确、和文字排版一体化
- 体会是 LaTeX 学习曲线较陡, 但一旦配置好模板, 排版质量和效率远超 Word/Pages

- 在本次 LaTeX 项目中第一次实战使用 Makefile 管理编译流程
- 写法上：定义变量 (MAIN, SRC, PDF)、声明依赖 (\$(PDF): \$(SRC))、编写规则 (latexmk -xelatex)
- 理解了 .PHONY 的作用——区分文件名和目标名，避免 make clean 和 clean 文件冲突
- make 一键编译、make clean 清理辅助文件、make distclean 彻底清除——比手动敲 xelatex 三次优雅得多
- 对 CMake 也有了初步了解：它是 Makefile 的高层抽象，通过 CMakeLists.txt 自动生成跨平台的构建脚本
- 总结下来：小项目用 Makefile 足够，跨平台大工程才需要 CMake——现在两个都会用了

课程感想

会用与精通之间

- 会几条命令、能装个软件，和真正精通 Linux 之间，差距比想象中大得多
- 比如课程中学了 `tr`，看起来只是字符替换的小工具——但它支持 POSIX 字符类、正则组合、管道串联，和 `grep`、`sort`、`uniq` 配合起来可以替代很多几十行的脚本
- 又比如 Vim：会 `h/j/k/l` 算是入门，但宏录制、文本对象、寄存器操作——这些真正提升效率的功能，需要大量刻意练习
- Linux 的每个工具都像冰山，表面看起来简单，水下的深度需要不断探索和积累
- 好的工具真的很多——`jq`、`tr`、`grep`、`sed`、`awk`——但如何把它们组合成高效的 `workflow`，不是学一两个命令就能做到的，需要持续打磨

- 这门课的学习过程中，AI 工具带来了很大的便利
- 写 Shell 脚本时，遇到不熟悉的语法就丢给 Claude Code 或 opencode 解释，几乎立即能得到带示例的说明——比翻 man 手册高效太多
- 在写 cal.sh 计算 π 时，bc 的精度参数和 Ramanujan 公式的实现细节，AI 可以帮忙快速验证逻辑，减少了很多试错成本
- opencode 尤其在 LaTeX 部分发挥了作用：修改 Beamer 主题、调整排版、写 Makefile——AI 协助下从零到完成一份 PDF 相当顺畅
- openclaw 等工具在处理批量文件操作时节省了大量手工劳动，比如批量重命名、格式化代码

- 但更重要的是 AI 没有替代思考：最终的理解、判断、决策——比如什么时候用管道什么时候写脚本、哪种排版更合适——还是需要自己来做
- AI 给出方案后，仍然需要人去评估是否正确、是否适合当前场景——批判性思维比任何时候都重要
- 未来这些工具只会越来越强，持续尝试新事物、更新自己的工具链，应该是每个技术从业者的基本素养
- 一个很深的体会是：AI 时代，知道问什么比知道怎么做是一个技术细节更重要——方向对了，AI 能帮你跑得更快

课程建议

- 使用开发容器（Dev Container）统一实验环境：
 - 目前学生各自安装 VirtualBox + Ubuntu，版本和配置差异会导致各种环境问题
 - 建议提供一个预配置的 Dev Container（基于 VS Code / Cursor 的 devcontainer.json），学生一键启动即可获得完全一致的 Ubuntu 环境
 - 容器内预装好 apt 镜像源、Zsh/oh-my-zsh、Vim/Neovim 等课程工具，省去作业 02 的大量配置时间
 - 保证所有学生在同一套环境下实验，减少「在我机器上能跑」的环境差异问题
- 增加 VCS 版本控制系统介绍：
 - SVN：了解集中式版本控制的原理，理解与 Git 的差异
 - Git：分布式版本控制，建议第一次作业就引入，逐步深入到分支管理、rebase -i、冲突解决
 - jj (Jujutsu)：兼容 Git 仓库的新一代工具，操作更直觉化——值得关注

用 PR 替代 ZIP 提交

- 课程提供一个作业仓库，每次作业作为独立目录；学生 fork 后在自己的分支上完成
- 提交时发起 Pull Request，助教在 PR 页面直接进行 Code Review——批注具体代码行、提出修改建议
- 学生根据 review 意见修改后 push 更新，迭代直到通过后 merge——这本身就是业界标准的工作流程
- 这套流程让学生从第一门课起就习惯版本控制、Code Review 和协作开发，比任何单独的 Git 练习课都有效

谢谢